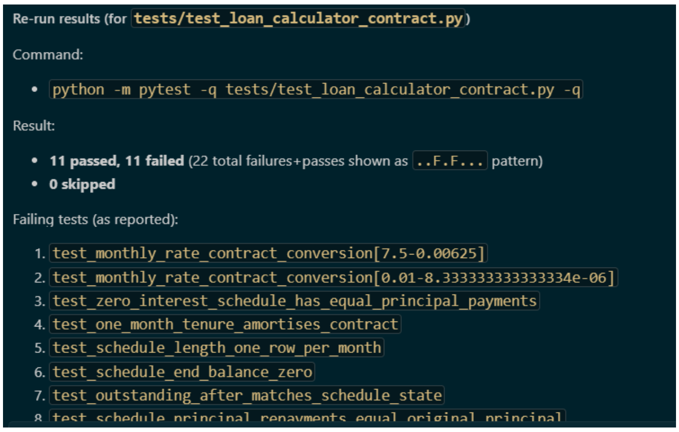
Assignment P1 - Documentation

1. Test Cases:

Initial Run:



Re-run results (for `tests/test_loan_calculator_contract.py`)

Command:

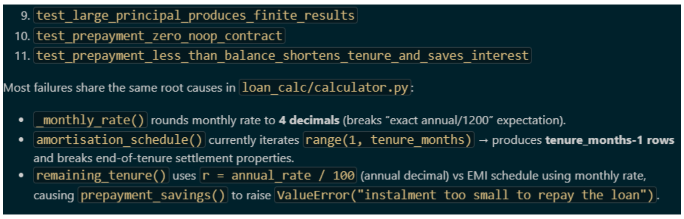
- `python -m pytest -q tests/test_loan_calculator_contract.py -q`

Result:

- **11 passed, 11 failed** (22 total failures+passes shown as `..F.F...` pattern)
- **0 skipped**

Failing tests (as reported):

1. `test_monthly_rate_contract_conversion[7.5-0.00625]`
2. `test_monthly_rate_contract_conversion[0.01-8.333333333333334e-06]`
3. `test_zero_interest_schedule_has_equal_principal_payments`
4. `test_one_month_tenure_amortises_contract`
5. `test_schedule_length_one_row_per_month`
6. `test_schedule_end_balance_zero`
7. `test_outstanding_after_matches_schedule_state`
8. `test_schedule_principal_repayments_equal_original_principal`

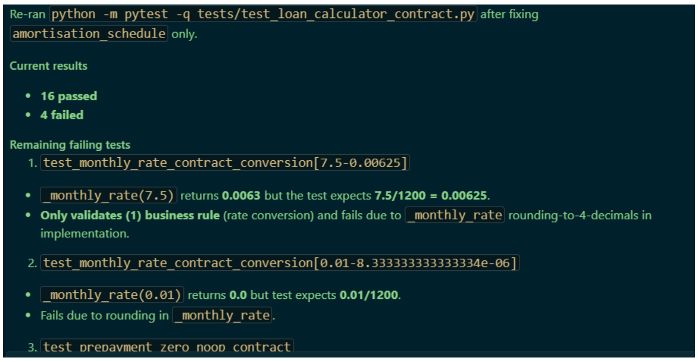


9. `test_large_principal_produces_finite_results`
10. `test_prepayment_zero_noop_contract`
11. `test_prepayment_less_than_balance_shortens_tenure_and_saves_interest`

Most failures share the same root causes in `loan_calc/calculator.py`:

- `_monthly_rate()` rounds monthly rate to **4 decimals** (breaks “exact annual/1200” expectation).
- `amortisation_schedule()` currently iterates `range(1, tenure_months)` → produces **tenure_months-1 rows** and breaks end-of-tenure settlement properties.
- `remaining_tenure()` uses `r = annual_rate / 100` (annual decimal) vs EMI schedule using monthly rate, causing `prepayment_savings()` to raise `ValueError("instalment too small to repay the loan")`.

After 1st Bug Fix:



Re-ran `python -m pytest -q tests/test_loan_calculator_contract.py` after fixing `amortisation_schedule` only.

Current results

- **16 passed**
- **4 failed**

Remaining failing tests

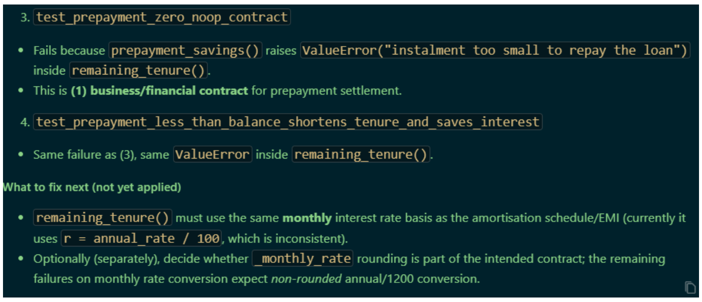
1. `test_monthly_rate_contract_conversion[7.5-0.00625]`

- `_monthly_rate(7.5)` returns **0.0063** but the test expects **7.5/1200 = 0.00625**.
- **Only validates (1) business rule** (rate conversion) and fails due to `_monthly_rate` rounding-to-4-decimals in implementation.

2. `test_monthly_rate_contract_conversion[0.01-8.333333333333334e-06]`

- `_monthly_rate(0.01)` returns **0.0** but test expects **0.01/1200**.
- Fails due to rounding in `_monthly_rate`.

3. `test_prepayment_zero_noop_contract`



3. `test_prepayment_zero_noop_contract`

- Fails because `prepayment_savings()` raises `ValueError("instalment too small to repay the loan")` inside `remaining_tenure()`.
- This is **(1) business/financial contract** for prepayment settlement.

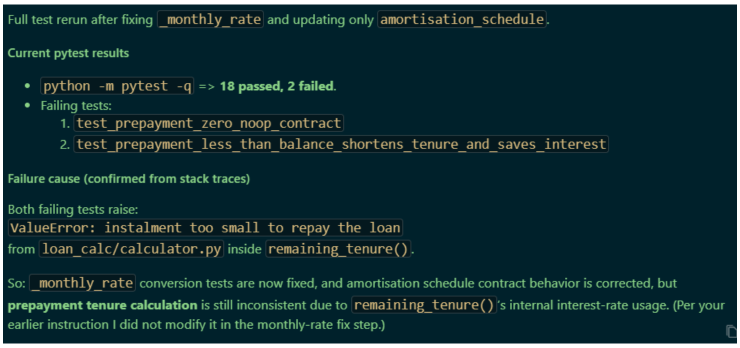
4. `test_prepayment_less_than_balance_shortens_tenure_and_saves_interest`

- Same failure as (3), same `ValueError` inside `remaining_tenure()`.

What to fix next (not yet applied)

- `remaining_tenure()` must use the same **monthly** interest rate basis as the amortisation schedule/EMI (currently it uses `r = annual_rate / 100`, which is inconsistent).
- Optionally (separately), decide whether `_monthly_rate` rounding is part of the intended contract; the remaining failures on monthly rate conversion expect *non-rounded* annual/1200 conversion.

After 2nd Bug Fix:



Full test rerun after fixing `_monthly_rate` and updating only `amortisation_schedule`.

Current pytest results

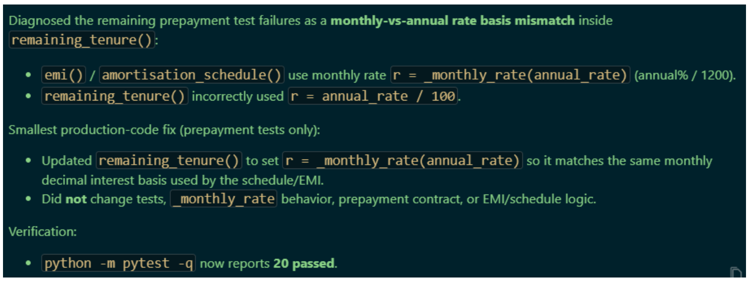
- `python -m pytest -q` => **18 passed, 2 failed**.
- Failing tests:
    1. `test_prepayment_zero_noop_contract`
    2. `test_prepayment_less_than_balance_shortens_tenure_and_saves_interest`

Failure cause (confirmed from stack traces)

Both failing tests raise:
`ValueError: instalment too small to repay the loan`
from `loan_calc/calculator.py` inside `remaining_tenure()`.

So: `_monthly_rate` conversion tests are now fixed, and amortisation schedule contract behavior is corrected, but **prepayment tenure calculation** is still inconsistent due to `remaining_tenure()`'s internal interest-rate usage. (Per your earlier instruction I did not modify it in the monthly-rate fix step.)

After 3rd Bug Fix:



Diagnosed the remaining prepayment test failures as a **monthly-vs-annual rate basis mismatch** inside `remaining_tenure()`:

- `emi()` / `amortisation_schedule()` use monthly rate `r = _monthly_rate(annual_rate)` (annual% / 1200).
- `remaining_tenure()` incorrectly used `r = annual_rate / 100`.

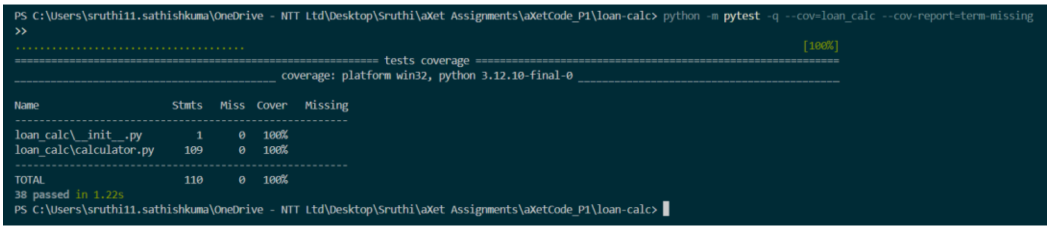
Smallest production-code fix (prepayment tests only):

- Updated `remaining_tenure()` to set `r = _monthly_rate(annual_rate)` so it matches the same monthly decimal interest basis used by the schedule/EMI.
- Did **not** change tests, `_monthly_rate` behavior, prepayment contract, or EMI/schedule logic.

Verification:

- `python -m pytest -q` now reports **20 passed**.

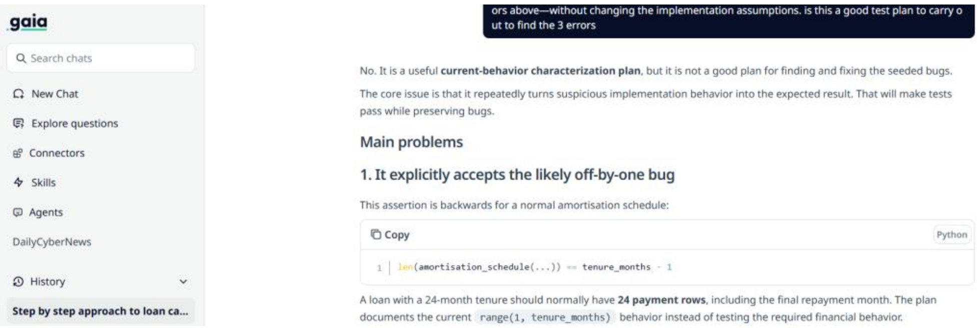
2. Test Coverage Results:



```
PS C:\Users\sruthi11.sathishkuma\OneDrive - NTT Ltd\Desktop\Sruthi\aXet Assignments\aXetCode_P1\loan-calc> python -m pytest -q --cov=loan_calc --cov-report=term-missing
>>
......................................                                                                          [100%]
================================================================ tests coverage ================================================================
______________________________________________ coverage: platform win32, python 3.12.10-final-0 _______________________________________________

Name                        Stmts   Miss  Cover   Missing
---------------------------------------------------------
loan_calc\__init__.py           1      0   100%
loan_calc\calculator.py       109      0   100%
---------------------------------------------------------
TOTAL                         110      0   100%
38 passed in 1.22s
PS C:\Users\sruthi11.sathishkuma\OneDrive - NTT Ltd\Desktop\Sruthi\aXet Assignments\aXetCode_P1\loan-calc> █
```

3. A moment when AI results was inaccurate and Plugin was prompted to revise its answer:



ors above—without changing the implementation assumptions. is this a good test plan to carry o ut to find the 3 errors

No. It is a useful **current-behavior characterization plan**, but it is not a good plan for finding and fixing the seeded bugs.

The core issue is that it repeatedly turns suspicious implementation behavior into the expected result. That will make tests pass while preserving bugs.

Main problems

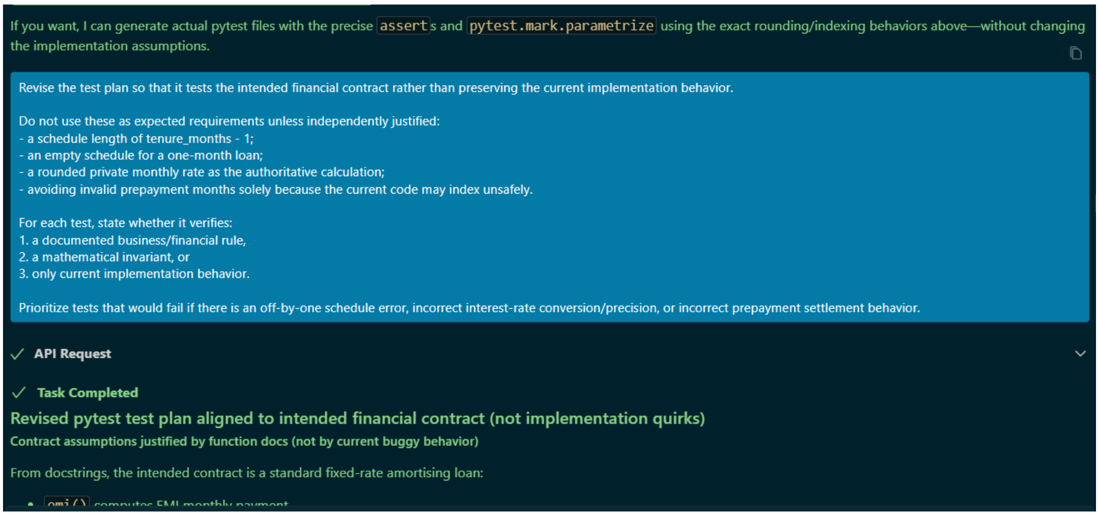
1. It explicitly accepts the likely off-by-one bug

This assertion is backwards for a normal amortisation schedule:

```python
len(amortisation_schedule(...)) == tenure_months - 1
```

A loan with a 24-month tenure should normally have **24 payment rows**, including the final repayment month. The plan documents the current `range(1, tenure_months)` behavior instead of testing the required financial behavior.

Response from .gaia when the initial test cases generated by aXet.plugin were passed to it for review.



If you want, I can generate actual pytest files with the precise `assert`s and `pytest.mark.parametrize` using the exact rounding/indexing behaviors above—without changing the implementation assumptions.

Revise the test plan so that it tests the intended financial contract rather than preserving the current implementation behavior.

Do not use these as expected requirements unless independently justified:
- a schedule length of tenure_months - 1;
- an empty schedule for a one-month loan;
- a rounded private monthly rate as the authoritative calculation;
- avoiding invalid prepayment months solely because the current code may index unsafely.

For each test, state whether it verifies:
1. a documented business/financial rule,
2. a mathematical invariant, or
3. only current implementation behavior.

Prioritize tests that would fail if there is an off-by-one schedule error, incorrect interest-rate conversion/precision, or incorrect prepayment settlement behavior.

Revised pytest test plan aligned to intended financial contract (not implementation quirks)

Contract assumptions justified by function docs (not by current buggy behavior)

From docstrings, the intended contract is a standard fixed-rate amortising loan:

- `emi()` computes EMI monthly payment

Prompt sent to aXet.plugin to regenerate test cases that are more focused on finding the bugs out.